Fraud Detection in Credit Card Transactions

Project completion report | Ekundayo Monday Edeki | github.com/edeki101/credit-card-fraud-detection

Introduction

Card fraud causes large financial losses, but fraudulent transactions are extremely rare, so naive models that simply predict “legitimate” look accurate while catching nothing. This project builds an end-to-end fraud detection pipeline on the public Kaggle credit card dataset: it compares two unsupervised anomaly detectors with a supervised XGBoost classifier, deals with the class imbalance, evaluates the models with metrics suited to rare events, and wraps the best model in a Streamlit web interface.

Abstract

The dataset holds 284,807 transactions by European cardholders, 492 of them fraudulent. After removing 1,081 duplicate rows, 283,726 transactions remained, 473 of them fraud (0.17%), and a stratified 80/20 split gave 226,980 training and 56,746 test transactions (95 frauds). Isolation Forest and Local Outlier Factor were fitted without labels; XGBoost was trained on SMOTE-balanced data. On the held-out test set XGBoost reached ROC-AUC 0.964 and average precision 0.766. With a decision threshold chosen on a separate validation set it caught 63 of 95 frauds (66% recall) with only 6 false alarms (91% precision, F1 0.77). Isolation Forest scored ROC-AUC 0.940 but average precision of only 0.13; Local Outlier Factor scored 0.692.

Tools Used

Python 3; **Pandas / NumPy** (data handling); **Scikit-Learn** (Isolation Forest, Local Outlier Factor, scaling, metrics); **imbalanced-learn** (SMOTE); **XGBoost** (classifier); **Matplotlib** (ROC, precision-recall, confusion matrix, feature importance); **Jupyter / Google Colab** (notebook); **Streamlit** (prediction dashboard); **Git/GitHub** (delivery).

Steps Involved in Building the Project

- **Load the data.** The Kaggle credit card dataset: 28 anonymized PCA features (V1–V28), Amount, Time and the fraud label.
- **Preprocess.** Removed 1,081 duplicates; applied log(1+x) and standardization to Amount; dropped Time; split 80/20 with stratification so the test set keeps the real fraud rate.
- **Anomaly detection.** Isolation Forest (200 trees) and Local Outlier Factor (20 neighbours, fitted on a 30,000-row sample for speed) were trained without labels; labels were used only to score them.
- **Balance the data.** A validation set (15% of training data) was held out first, then SMOTE created synthetic frauds in the remaining training data until the classes were equal. SMOTE was never applied to validation or test data.
- **Train XGBoost.** Gradient-boosted trees (depth 6) with early stopping on validation average precision (stopped at iteration 49).
- **Evaluate.** ROC and precision-recall curves, confusion matrices, and a decision threshold (0.98) chosen to maximize F1 on the validation set, not the test set.
- **Dashboard.** A Streamlit app checks a single transaction, scores an uploaded CSV file and shows the model performance charts; it is ready to deploy as a web app.

Table 1. Test-set results (56,746 transactions, 95 fraud). PR-AUC = average precision.

Model	ROC-AUC	PR-AUC	Precision	Recall	F1
Isolation Forest	0.940	0.127	0.224	0.253	0.238
Local Outlier Factor	0.692	0.009	0.021	0.021	0.021
XGBoost (threshold 0.5)	0.964	0.766	0.219	0.832	0.347
XGBoost (tuned threshold 0.98)	**0.964**	**0.766**	**0.913**	**0.663**	**0.768**

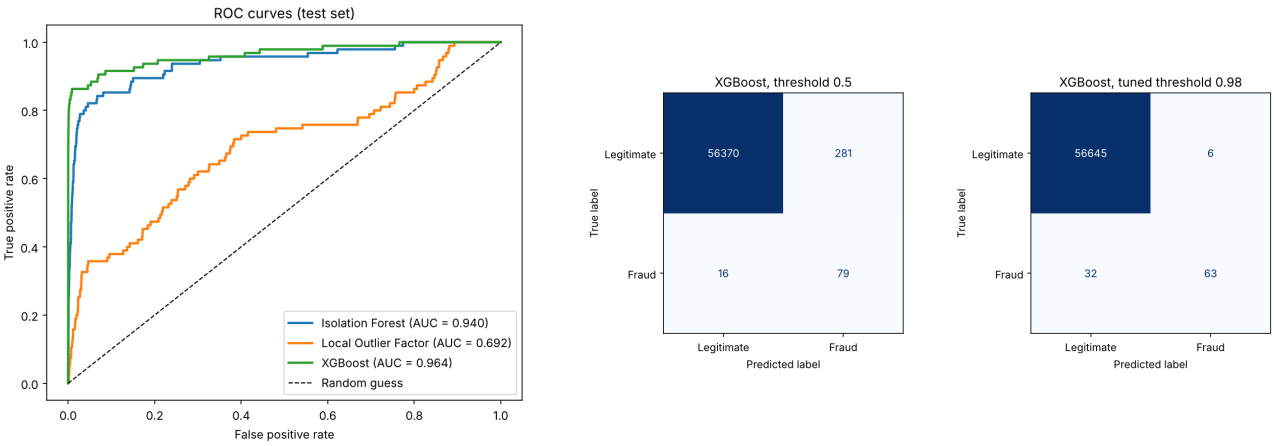


Figure 1. Left: ROC curves of the three models on the test set. Right: XGBoost confusion matrices at the default 0.5 threshold and at the tuned threshold.

Conclusion

The supervised XGBoost classifier clearly outperformed both unsupervised detectors, which is expected because it learns from labelled fraud, while Isolation Forest and Local Outlier Factor are useful mainly when no labels exist. Accuracy is meaningless for this problem (a model that never flags fraud scores about 99.8%), so ROC-AUC, average precision, precision and recall were used. The threshold controls the trade-off: at 0.5 the model catches 83% of frauds but raises 281 false alarms; at the tuned threshold it raises only 6 but misses 32 frauds. The threshold is high (0.98) because SMOTE shifts the predicted probabilities. Limitations: only 95 frauds are in the test set, so the estimates are uncertain; the data is anonymized and covers two days of 2013; and the split is random rather than chronological, so real-world performance would likely be lower. Next steps: time-based validation, a cost-based threshold, probability calibration and periodic retraining. Educational project, not a production fraud system.